

Welcome to Week 4 - LangChain and LangGraph

Lab 1: Abstraction Levels and the building blocks



The LangChain docs

The documentation lives at <https://docs.langchain.com/oss/python/>. It is worth a browse. Watch out for older 0.x material that still floats around the web, because the modern API is quite different.

The four levels of abstraction

LangChain and LangGraph form 4 levels of abstraction, with each one built on the ones before. The terminology is confusing because 'LangChain' appears in a few places..

Layer	Packages	What it gives you	What you control
1. Building blocks	langchain-core + langchain-openai	chat models, the @tool decorator, messages, structured output	everything, including the tool loop by hand
2. Orchestration	langgraph	a graph of steps, with state, memory and checkpointing	the control flow (you design the graph)
3. Agent	langchain (create_agent)	the standard agent loop, prebuilt	just model, tools and a prompt
4. Harness	deepagents (create_deep_agent)	an opinionated harness with planning, sub-agents and a filesystem	your intent

Order of play for this week:

DAY 1 (today): The building blocks

DAY 2: LangGraph

DAY 3: LangChain create_agent

DAY 4: Deep Agents

DAY 5: The Sidekick project

Today is Layer 1: the building blocks

This is where LangChain began: an abstraction layer, not unlike LiteLLM - but a more heavyweight version.

```
In [1]: import os
        from dotenv import load_dotenv
        from langchain_openai import ChatOpenAI
        from langchain_core.messages import HumanMessage, SystemMessage,
        ToolMessage
        from langchain_core.tools import tool
        from pydantic import BaseModel, Field, SecretStr

        load_dotenv(override=True)
```

```
Out[1]: True
```

A first model call

`ChatOpenAI` is the abstraction around OpenAI calls, and there are similar packages for other LLM providers.

We call `invoke` with a prompt; this is a key method in `LangChain`.

```
In [2]: api_key_str = os.environ.get("OPENROUTER_API_KEY")

        llm = ChatOpenAI(
            model="openrouter/free",
            api_key=SecretStr(api_key_str) if api_key_str else None,
            base_url="https://openrouter.ai/api/v1"
        )

        message = "In 1 sentence, what does it mean for an AI Agent to be
        autonomous"

        reply = llm.invoke(message)

        print(reply.content)
```

An autonomous AI agent is one that can independently perceive its environment, reason through complex goals, and execute the necessary sequence of actions to achieve an objective without requiring step-by-step human intervention.

Streaming

For a live, token by token feel, swap `invoke` for `stream` and loop over the chunks

```
In [3]: for chunk in llm.stream("Tell me a two line poem about autonomous agents"):
        print(chunk.content, end="", flush=True)
```

```
In silent code they wander, unseen,
Agents choose their own horizon.
```

Messages

LangChain comes with abstractions around `SystemMessage`, `HumanMessage`, `AIMessage`, although you can use the usual list-of-dicts instead.

```
In [4]: messages = [
        SystemMessage("You are a terse assistant who answers in exactly five words."),
        HumanMessage("What is the capital of France?"),
    ]

    print(llm.invoke(messages).content)

    # The exact same call using plain dictionaries, the format you already know
    messages_as_dicts = [
        {"role": "system", "content": "You are a terse assistant who answers in exactly five words."},
        {"role": "user", "content": "What is the capital of France?"},
    ]
    print(llm.invoke(messages_as_dicts).content)
```

```
The capital city is Paris
France's capital city is Paris.
```

Tools with the `@tool` decorator

A tool is a Python function the model is allowed to call. The modern way to make one is the `@tool` decorator. Your docstring becomes the description the model reads, and your type hints become the argument schema, just like `@function_tool` with OpenAI Agents SDK.

This replaces the older `Tool(...)` wrapper from earlier versions of LangChain.

```
In [5]: @tool
def get_share_price(symbol: str) -> float:
    """Return the current share price for a given ticker symbol."""
    fake_prices = {"AAPL": 241.5, "GOOG": 168.2, "AMZN": 198.0}
    return fake_prices.get(symbol.upper(), 0.0)

print("name:", get_share_price.name)
print("description:", get_share_price.description)
print("args:", get_share_price.args)
print("called directly:", get_share_price.invoke({"symbol": "AAPL"}))

name: get_share_price
description: Return the current share price for a given ticker
symbol.
args: {'symbol': {'title': 'Symbol', 'type': 'string'}}
called directly: 241.5
```

Giving tools to the model

This is a bit clunky; redemption will come in Day 3 when we use the `create_agent` features,

For now, we have to write the loop ourselves, rather like we did in Week 1.

The first step is to bind the tools to the model with `bind_tools`. Now when we invoke, the model may come back not with an answer but with a request to run a tool. That request shows up in `.tool_calls`.

```
In [6]: llm_with_tools = llm.bind_tools([get_share_price])

response = llm_with_tools.invoke("What is the share price of Amazon?")
print("content:", repr(response.content))
print("tool_calls:", response.tool_calls)

content: ''
tool_calls: [{'name': 'get_share_price', 'args': {'symbol': 'AMZN'},
'id': 'call_7cdfde4ebb294785a5ce42eb', 'type': 'tool_call'}]
```

Running the tool loop by hand

So now we need to write a little loop, quite similar to Week 1

```
In [7]: # Start the conversation and keep the model's tool request in the history
conversation = [HumanMessage("What is the share price of Amazon?")]
ai_message = llm_with_tools.invoke(conversation)
conversation.append(ai_message)

# Run each requested tool and add its result as a ToolMessage
for call in ai_message.tool_calls:
    if call["name"] == "get_share_price":
        result = get_share_price.invoke(call["args"])
        conversation.append(ToolMessage(content=str(result),
tool_call_id=call["id"]))

# Invoke again, now that the model can see the tool result
final = llm_with_tools.invoke(conversation)
print(final.content)
```

The current share price of Amazon (AMZN) is \$198.00.

Structured output

Similar to OpenAI Agents SDK, we can require the model to respond with a Pydantic subclass

```
In [8]: class Company(BaseModel):
        name: str = Field(description="The company name")
        ticker: str = Field(description="The stock ticker symbol")
        founded_year: int = Field(description="The year the company was
        founded")

        structured_llm = llm.with_structured_output(Company)

        company = structured_llm.invoke("Tell me about Amazon the technology
        company")
        print(company)
        print("Just the ticker:", company.ticker)
```

```
name='websearch' ticker='amazon.com' founded_year=1994
Just the ticker: amazon.com
```

That is Layer 1

It's like a more rich and more involved version of LiteLLM.



Exercise

Write a second tool of your own, perhaps one that looks up a fictional weather report for a city. Bind both tools to the model and ask a question that needs both, then run the tool loop by hand until the model gives a final answer. It will feel more gritty than OpenAI Agents SDK or CrewAI.

We will fix that by Day 3!

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).